

Day 54 -- Line-by-Line Syntax Explanation

day54_batchnorm_in_cnns.py, segment by segment | MD Mutasim Billah

1. spatial_batchnorm_forward/backward -- the New Core Helper

forward: per-channel stats pooled over batch AND spatial axes

```
mu = Z.mean(axis=(0, 2, 3), keepdims=True) # per-channel mean over (batch, H, W)
var = Z.var(axis=(0, 2, 3), keepdims=True) # per-channel variance, same pooling
Zhat = (Z - mu) / np.sqrt(var + eps) # normalize to zero mean, unit variance
out = gamma * Zhat + beta # learned per-channel rescale + shift
```

axis=(0,2,3) is the only structural change from Day 40's dense batch norm, which used axis=0 alone. gamma and beta have shape (1, C, 1, 1) so they broadcast against Z's (m, C, H, W) shape -- one learned scale/shift per channel, shared across every spatial position and every example in the batch.

backward: the /m fix a real gradient-check failure caught

```
N = m * H * W # total values pooled per channel for the BN statistics themselves
dgamma = np.sum(dout * Zhat, axis=(0,2,3), keepdims=True) / m # note the /m -- NOT dividing
by m was the bug
dbeta = np.sum(dout, axis=(0,2,3), keepdims=True) / m # same /m, same reason
dZ = dZhat * std_inv + dvar * 2*(Z-mu)/N + dmu/N # this part uses N (=m*H*W), a SEPARATE,
correct use
```

Two different "totals" appear in this function and must not be confused: $N=m*H*W$ is how many values were pooled to compute mu/var themselves (a property of the batchnorm MATH, independent of this codebase's conventions), while the /m on dgamma/dbeta is this codebase's OWN convention that every parameter gradient (dW, db, and now dgamma/dbeta) is scaled to match cross_entropy_loss's mean-over-the-batch normalization -- the same /m conv2d_backward_mc and the FC layers already apply to their own dW/db.

2. plainblock_bn_forward/backward and resblock_bn_forward/backward

conv -> BN -> ReLU, not conv -> ReLU -> BN

```
z1 = conv2d_forward_mc(x, w1, b1, pad=pad) # first conv
bn1, bn1_cache = spatial_batchnorm_forward(z1, gamma1, beta1) # normalize BEFORE the relu
a1 = relu(bn1) # relu applied to the NORMALIZED value, not z1 directly
```

This ordering -- conv, then BN, then ReLU -- matches every real BatchNorm2d placement, including ResNet's. Backward mirrors it: $dbn1 = da1 * drelu(bn1)$ undoes the ReLU using bn1 (the post-BN value), not z1, since that's what the ReLU was actually applied to.

resblock_bn_forward: the identity shortcut adds around the SECOND BN, not after it

```
z2 = conv2d_forward_mc(a1, w2, b2, pad=pad)
bn2, bn2_cache = spatial_batchnorm_forward(z2, gamma2, beta2) # normalize, but NO relu yet
out = relu(bn2 + x) # add the shortcut to the NORMALIZED bn2, then relu
```

This is the real ResNet basic block's exact structure: the second BN's output is added to the identity shortcut BEFORE the final ReLU, not after. Note x itself is NOT normalized -- it's whatever came out of the previous block/stem's own ReLU. This is exactly the asymmetry the module docstring identifies as a likely cause of ResNetBNConvNet's noisier training: $bn2$ is freshly renormalized every forward pass, but x is not, so their relative scales aren't controlled the way BN alone controls a single branch.

3. demo_gradient_flow_by_depth() -- Three Networks, One Fixed Batch

the ratio metric extended from Day 53 to three networks

```
ratios = [stem_row[k] / max(deep_row[k], 1e-12) for k in (1, 2, 3)] # plain, BN-only, BN+res
ratios
print(f"...plain: {ratios[0]:.4f}, BN-only: {ratios[1]:.4f}, BN+residual:
{ratios[2]:.4f}...")
```

Day 53 computed this ratio for 2 networks; today's version generalizes the same computation across 3, reading from whichever index (plain networks' `block_grads` tuples are 4 elements; BN networks' are 8) holds each network's second-conv weight gradient, since `plainblock_bn_backward/resblock_bn_backward` return more values per block (weight, bias, gamma, AND beta gradients) than the no-BN blocks do.